

Autonomous Factor Research Loop

Implementation Instruction and Research Specification

Goal. Build a reproducible system that proposes, constructs, evaluates, rejects, and iterates on candidate equity factors using the NYSE price/fundamental data and the general-news dataset. The deliverable is the research process, not one hand-crafted factor.

Core design principle

Use the LLM for hypothesis generation and mutation, but keep factor construction, point-in-time checks, backtesting, selection, and experiment logging deterministic. The agent may propose; the research engine decides.

1. System Architecture

Data -> Feature Library -> Candidate Generator -> Factor Builder -> Integrity Gate -> Walk-Forward Evaluator -> Selector -> Mutator -> Next Generation

Recommended module boundaries:

Module	Responsibility
data_loader.py	Load, align, clean, and timestamp raw datasets.
feature_engine.py	Create reusable price, fundamental, and news primitives.
factor_dsl.py	Represent factors as constrained expressions; no arbitrary code execution.
generator.py	Create seed candidates and LLM-guided mutations.
integrity.py	Reject look-ahead, invalid timestamps, missing fields, and suspicious constructions.
evaluator.py	Compute cross-sectional IC, portfolio metrics, fold stability, and costs.
selector.py	Apply hard gates, redundancy checks, and survivor logic.
mutator.py	Generate children from survivors.
experiment_log.py	Persist every candidate, metric, decision, and rejection reason.

2. Data and Point-in-Time Rules

- Signal at date t may only use information that would have been available by t . Prediction target must begin after the signal timestamp.
- Fundamentals must be aligned to an availability date, not blindly to fiscal-period end. If a release timestamp is unavailable, impose a conservative reporting lag and document it.
- News must be timestamped and aggregated only from articles published before the signal cutoff.
- Apply corporate-action-adjusted prices where appropriate; record missing-data policy and universe filters explicitly.
- The integrity checker should fail closed: suspicious timing or unavailable fields cause rejection rather than silent repair.

Required demonstration

Include at least one intentionally or naturally discovered bad factor that looks excellent before the integrity check but is rejected for look-ahead or timing contamination. Show the naive result versus the corrected result.

3. Factor Search Space

Use a **constrained factor language**. Keep the search expressive enough to discover structure, but small enough to audit and reproduce.

Family	Examples	Allowed transforms
Price / market	1d/5d/20d/60d return, volatility, volume shock, range, distance to high	rank, z-score, delta, rolling mean/std, EWMA, sign, winsorize
Fundamental	book-to-price, earnings yield, ROE, margin, leverage, growth, asset growth	level, change, cross-sectional rank/z-score
News	sentiment, negative share, news volume, dispersion, topic counts, novelty	rolling sum/mean, EWMA, z-score, regime bucket
Interactions	price x news regime, value x sentiment, momentum / volatility	pairwise interaction, residualization, simple linear combination

Example DSL candidate:

```
base = return_20d
transform = cross_sectional_rank
normalizer = volatility_20d
interaction = news_sentiment_5d
direction = +1
complexity = 4 operators
```

4. Candidate Generation and Iteration

Recommended approach: hybrid search. Start rule-based and reproducible, then let the LLM propose bounded mutations based on prior evidence.

1. Generation 0 - seed 20-40 simple factors from price, fundamentals, and news.
2. Generation 1 - mutate surviving windows, smoothing, normalization, and direction.
3. Generation 2 - add simple cross-family interactions or market/news regime conditioning.
4. Generation 3 - test residual/incremental information and combine complementary survivors.
5. Final - freeze the system and evaluate the selected factor set once on the untouched holdout.

LLM input should contain evidence, not raw unrestricted data:

```
Parent: Momentum20
Validation IC: 0.021
Net Sharpe: 0.92
Turnover: 1.3x
Weakness: poor performance in high-volatility negative-news periods
Task: propose <= 3 mutations using only the allowed DSL operators.
```

5. Evaluation Methodology

Do not optimize on one full-sample Sharpe ratio. A candidate should survive only when it shows predictive power, economic usefulness, stability, and novelty out of sample.

5.1 Validation scheme

```
Research universe: 2010-2015 only
Walk-forward folds:
  2010-2012 -> validate 2013
  2010-2013 -> validate 2014
  2010-2014 -> validate 2015
Final untouched holdout: 2016, evaluated once after the loop is frozen
```

5.2 Cross-sectional predictive evidence

- Daily rank IC: Spearman correlation between factor_{i,t} and future return_{i,t+1:t+h}.
- Track mean IC, IC volatility, IC information ratio, hit rate, and yearly/fold sign consistency.
- Require performance to survive across multiple folds rather than one favorable period.

5.3 Portfolio evidence

- Construct a transparent long-short portfolio from cross-sectional ranks (for example, top quintile minus bottom quintile).
- Track annualized return, net Sharpe, drawdown, turnover, and return per unit of turnover.
- Use a simple transaction-cost assumption and report sensitivity to a higher-cost scenario.
- If metadata permit, neutralize obvious unwanted exposures; otherwise keep the portfolio dollar-neutral and state the limitation.

5.4 Survivor logic

Use hard gates before ranking. Avoid one arbitrary mega-score that can hide a fatal weakness.

Gate	Example rule
Integrity	No timestamp violation, unavailable data, or target leakage.
Predictive sign	Positive mean IC and same sign in at least 2 of 3 validation folds.
Economic value	Positive net portfolio performance after costs; avoid candidates driven by one month/year.
Stability	No catastrophic subperiod and no extreme sensitivity to one parameter choice.
Complexity	Prefer fewer operators when performance is similar.
Novelty	Reject near-duplicates of an existing survivor.

6. Redundancy, Incremental Alpha, and Multiple Testing

- If two candidate factors are highly correlated (for example, > 0.85 cross-sectionally or in portfolio returns), keep the simpler or more stable one.
- For a child factor, test incremental information relative to its parent. Residualize the child on the parent and evaluate residual IC.
- Track the total number of candidates tested. Autonomous search creates selection bias even when each individual backtest is technically out of sample.
- Use the untouched holdout, fold consistency, complexity control, and a null/random benchmark to reduce false discoveries.

Optional but strong

Generate random or permuted pseudo-factors and compare the best searched candidate against the distribution of best results obtained under the null. This directly asks whether the discovered performance is unusual after accounting for the search process.

7. Experiment Log and Trajectory

Every tested factor must be auditable. Store the exact definition, parent, generation, metrics, decision, and reason.

Gen	Factor	Origin	IC	Sharpe	Turnover	Decision	Reason
0	Mom20	seed	2.1%	0.92	1.3	KEEP	stable
0	Mom5	seed	-0.2%	-0.10	3.8	KILL	no signal
1	Mom20/Vol20	mutation	2.8%	1.18	1.1	KEEP	more stable
1	Mom17	mutation	2.2%	0.94	1.4	KILL	redundant
2	Mom x News	interaction	3.1%	1.25	1.0	KEEP	incremental

Also produce a factor family tree. This should show which seed produced each child, which branches died, and why.

```
Momentum20
|-- Momentum10      [killed: unstable]
|-- Momentum40      [killed: weak]
|-- Momentum20 / Vol20 [survived]
    |-- x NewsSentiment [survived]
    |-- x NewsVolume   [killed: no incremental IC]
```

8. Final Combination

Keep the final model simple. Standardize surviving factors cross-sectionally and combine only complementary signals.

- Baseline: equal-weight average of standardized survivors.
- Optional: ridge regression fit only on the allowed training window and re-estimated walk-forward.
- Do not add a complex ML model unless it clearly improves out-of-sample evidence and remains interpretable.

9. Main Loop Pseudocode

```
history = []
survivors = seed_candidates()

for generation in range(N_GENERATIONS):
    candidates = propose_or_mutate(survivors, history)
    results = []

    for candidate in candidates:
        factor = build_from_dsl(candidate)
        integrity = validate_point_in_time(factor)
        if not integrity.ok:
            log(candidate, decision='KILL', reason=integrity.reason)
            continue

        metrics = evaluate_walk_forward(factor)
        novelty = compare_to_existing_survivors(factor)
        decision = apply_survivor_gates(metrics, novelty, candidate.complexity)
        log(candidate, metrics, decision)
        results.append((candidate, metrics, decision))

    survivors = select_survivors(results)

freeze_system()
run_final_holdout(year=2016)
```

10. One-Page Memo Structure

1. Evaluation philosophy - what counts as evidence of alpha and why.
2. Why the loop is resistant to lucky discoveries - walk-forward validation, holdout, stability, redundancy, complexity, and multiple-testing controls.
3. How the system detects and handles look-ahead and data-integrity failures.
4. What remains imperfect and what you would add with more time: point-in-time universe membership, stronger cost calibration, sector neutrality, richer entity mapping, and formal multiple-testing adjustments.

11. Deliverables

- A runnable research loop with configuration-driven factor search.
- A factor/feature library and constrained DSL.
- An experiment log containing every candidate and decision.
- Walk-forward evaluation results and final 2016 holdout results.
- A trajectory table and factor family tree.
- At least one explicit data-integrity/look-ahead rejection example.
- A concise README describing how to reproduce the run.

- A one-page memo explaining why the evaluation methodology is trustworthy.

Scope recommendation

Aim for roughly 100-300 total evaluated candidates, not an unbounded search. A disciplined system with clear evidence and rejection logic is more convincing than a huge agentic search that produces an implausibly high backtest Sharpe.

12. Implementation Order

1. Build the data alignment and point-in-time layer.
2. Implement the reusable feature library and DSL.
3. Implement walk-forward IC and portfolio evaluation.
4. Add integrity gates and experiment logging.
5. Create Generation 0 rule-based seeds and verify end-to-end reproducibility.
6. Add mutation and LLM-guided hypothesis generation.
7. Add redundancy/incremental-alpha tests and multiple-testing diagnostics.
8. Freeze the loop, run the holdout once, then write the memo and trajectory narrative.

13. Final Acceptance Checklist

- ☐ No factor uses information unavailable at the signal timestamp.
- ☐ 2016 is untouched until the research loop and selection rules are frozen.
- ☐ Every candidate has a reproducible definition and logged decision reason.
- ☐ Costs, turnover, stability, redundancy, and complexity are evaluated - not Sharpe alone.
- ☐ At least one failed integrity example is shown explicitly.
- ☐ The final narrative explains what the loop learned across generations, not just the final winners.